# Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: C0C25081/シールド機能

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests | Create pull request

1 commit | 1 file changed | 1 contributor

Commits on Nov 3, 2026

**Eキーで発動するシールド機能を追加**

c0c25081b8 committed 29 minutes ago — 0ec14b2

Showing 1 changed file with 62 additions and 48 deletions. Split | Unified

110 geme_base.py

```
19  19    COLOR_ENEMY = (255, 50, 50)    # 赤色（敵）
20  20    COLOR_WHITE = (255, 255, 255)  # 白色
21  21    COLOR_RED = (200, 0, 0)        # 強調用赤
    22  + COLOR_SHIELD = (0, 230, 255)   # シールド用シアン
22  23
23  24
24  25    class Player:
25      -     """プレイヤーキャラクターの移動、HP、描画を管理するクラス"""
    26  +     """プレイヤーの移動、HP、シールド（バリア）、描画を管理するクラス"""
26  27
27  28        def __init__(self, x: float, y: float) -> None:
28      -         """プレイヤーの初期位置やステータスを設定する"""
    29  +         """初期位置とステータスの設定"""
29  30            self.x: float = x
30  31            self.y: float = y
31  32            self.radius: int = 15
32  33            self.speed: float = 4.0
33  34            self.hp: int = 100
34  35            self.max_hp: int = 100
35      -         self.invincible_timer: int = 0  # 被弾後の無敵時間フレームカウンタ
    36  +         self.invincible_timer: int = 0  # 被弾後の無敵タイマー
    37  +
    38  +         # Eキーで展開するシールド（無敵バリア）関連変数
    39  +         self.shield_timer: int = 0      # シールド展開時間（フレーム）
```

```
                40 +          self.shield_cooldown: int = 0  # クールダウン時間（フレーム）
36              41
37              42        def update(self, keys: pygame.key.ScancodeWrapper) -> None:
38                -          """キー入力を受けて移動処理を行い、画面外への逸脱を防ぐ"""
                43 +          """移動入力とシールドタイマーの更新"""
39              44            dx: float = 0.0
40              45            dy: float = 0.0
41              46
42                -          # WASDキーによる入力判定
                47 +          # WASDキー入力
43              48            if keys[pygame.K_w]:
44              49                dy -= 1.0
45              50            if keys[pygame.K_s]:
49              54            if keys[pygame.K_d]:
50              55                dx += 1.0
51              56
52                -          # 斜め移動時の速度が速くならないように正規化（1 / √2 ≒ 0.7071）
                57 +          # Eキー入力でシールド発動（クールダウン中でない場合）
                58 +          if keys[pygame.K_e] and self.shield_cooldown == 0:
                59 +              self.shield_timer = 120      # 2秒間（120フレーム）無敵
                60 +              self.shield_cooldown = 1800  # 再使用まで30秒間（1800フレーム）のクールダウン
                61 +
                62 +          # 斜め移動時の速度補正
53              63            if dx != 0.0 and dy != 0.0:
54              64                dx *= 0.7071
55              65                dy *= 0.7071
56              66
57                -          # 位置の更新
                67 +          # 座標の更新
58              68            self.x += dx * self.speed
59              69            self.y += dy * self.speed
60              70
61                -          # 画面外に出ないように制限
                71 +          # 画面外逸脱防止
62              72            if self.x - self.radius < 0:
63              73                self.x = self.radius
64              74            elif self.x + self.radius > SCREEN_WIDTH:
69              79            elif self.y + self.radius > SCREEN_HEIGHT:
70              80                self.y = SCREEN_HEIGHT - self.radius
71              81
72                -          # 無敵タイマーのカウントダウン
                82 +          # タイマーの減算処理
                83 +          if self.shield_timer > 0:
                84 +              self.shield_timer -= 1
                85 +          if self.shield_cooldown > 0:
                86 +              self.shield_cooldown -= 1
73              87            if self.invincible_timer > 0:
74              88                self.invincible_timer -= 1
75              89
76              90        def draw(self, screen: pygame.Surface) -> None:
77                -          """プレイヤーを描画する（無敵時間中は点滅表現を行う）"""
78                -          # 無敵時間中は点滅させる（5フレームごとに表示・非表示を切り替え）
                91 +          """プレイヤーとシールドの描画"""
                92 +          # シールド展開中のエフェクト描画（水色の円枠）
                93 +          if self.shield_timer > 0:
                94 +              pygame.draw.circle(screen, COLOR_SHIELD, (int(self.x), int(self.y)),
                    self.radius + 8, 3)
                95 +
```

```
                96 +          # 被弾無敵時間中の点滅処理
79              97            if self.invincible_timer > 0 and (self.invincible_timer // 5) % 2 == 0:
80              98                return
81              99
82              100           pygame.draw.circle(screen, COLOR_PLAYER, (int(self.x), int(self.y)),
                    self.radius)
83              101           pygame.draw.circle(screen, COLOR_WHITE, (int(self.x), int(self.y)),
                    self.radius, 2)
84              102
85              103       def get_pos(self) -> tuple[float, float]:
86                  -         """プレイヤーの現在座標を返す"""
                    104 +         """現在座標の取得"""
87              105           return (self.x, self.y)
88              106
89              107       def get_hp(self) -> int:
90                  -         """プレイヤーの現在のHPを返す"""
                    108 +         """現在HPの取得"""
91              109           return self.hp
92              110
93              111       def take_damage(self, amount: int) -> None:
94                  -         """ダメージを受け、無敵時間を付与する"""
                    112 +         """ダメージ処理（シールド展開中は無効化）"""
                    113 +         # シールド展開中はダメージを受けない
                    114 +         if self.shield_timer > 0:
                    115 +             return
                    116 +
95              117           if self.invincible_timer == 0:
96              118               self.hp -= amount
97              119               if self.hp < 0:
98              120                   self.hp = 0
99                  -             self.invincible_timer = 60  # 1秒間（60フレーム）の無敵時間
                    121 +             self.invincible_timer = 60  # 被弾後1秒間の無敵
100             122
101             123
102             124   class Enemy:
103             125       """敵キャラクターの移動と描画を管理するクラス"""
104             126
105             127       def __init__(self, x: float, y: float) -> None:
106                 -         """敵の初期位置を設定する"""
107             128           self.x: float = x
108             129           self.y: float = y
109             130           self.radius: int = 12
110             131           self.speed: float = 2.2
111             132
112             133       def update(self, player_x: float, player_y: float) -> None:
113                 -         """プレイヤーの方向に向かって移動する"""
                    134 +         """プレイヤーに向かって追従移動"""
114             135           dx: float = player_x - self.x
115             136           dy: float = player_y - self.y
116             137           distance: float = math.hypot(dx, dy)
117             138
118                 -         # プレイヤーに向かって単位ベクトルを掛けて移動
119             139           if distance > 0:
120             140               dx /= distance
121             141               dy /= distance
122             142               self.x += dx * self.speed
123             143               self.y += dy * self.speed
124             144
```

```
125 145     def draw(self, screen: pygame.Surface) -> None:
126         -       """敵を描画する"""
    146     +       """敵の描画"""
127 147             pygame.draw.circle(screen, COLOR_ENEMY, (int(self.x), int(self.y)), self.radius)
128 148             pygame.draw.circle(screen, COLOR_WHITE, (int(self.x), int(self.y)), self.radius, 2)
129 149
130 150     def get_pos(self) -> tuple[float, float]:
131         -       """敵の現在座標を返す"""
132 151             return (self.x, self.y)
133 152
134 153
135 154 class Game:
136         -   """ゲーム全体の状態、時間、スコア、当たり判定を管理するクラス"""
    155     +   """ゲーム全体の管理クラス"""
137 156
138 157     def __init__(self) -> None:
139         -       """Pygameとゲーム変数を初期化する"""
140 158             pygame.init()
141 159             self.screen: pygame.Surface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
142 160             pygame.display.set_caption("SURVIVAL ARENA")
147 165             self.reset_game()
148 166
149 167     def reset_game(self) -> None:
150         -       """ゲームの状態を初期化またはリスタートする"""
    168     +       """ゲーム状態のリセット"""
151 169             self.player: Player = Player(float(SCREEN_WIDTH / 2), float(SCREEN_HEIGHT / 2))
152 170             self.enemies: list[Enemy] = []
153 171             self.elapsed_frames: int = 0
154 172             self.spawn_timer: int = 0
155         -       self.game_state: str = "PLAYING"  # "PLAYING", "GAME_OVER", "CLEAR"
    173     +       self.game_state: str = "PLAYING"
156 174
157 175     def run(self) -> None:
158         -       """メインゲームループを実行する"""
    176     +       """メインループ"""
159 177             running: bool = True
160 178             while running:
161         -               # イベント処理
162 179                 for event in pygame.event.get():
163 180                     if event.type == pygame.QUIT:
164 181                         running = False
165 182                     elif event.type == pygame.KEYDOWN:
166 183                         if event.key == pygame.K_ESCAPE:
167 184                             running = False
168 185                         elif event.key == pygame.K_r:
169         -                           # ゲームオーバーまたはクリア時にRキーでリスタート
170 186                             if self.game_state in ["GAME_OVER", "CLEAR"]:
171 187                                 self.reset_game()
172 188
173 189                 keys = pygame.key.get_pressed()
174 190
175 191                 if self.game_state == "PLAYING":
176         -                   # プレイヤーの更新
177 192                     self.player.update(keys)
```

```
178  193
179       -                # 時間経過のカウント
180  194                   self.elapsed_frames += 1
181  195                   survived_seconds = self.elapsed_frames // FPS
182  196                   if survived_seconds >= TIME_LIMIT:
183  197                       self.game_state = "CLEAR"
184  198
185       -                # 一定時間ごと（約1.5秒おごと）に敵を出現させる
186  199                   self.spawn_timer += 1
187  200                   if self.spawn_timer >= 90:
188  201                       self.spawn_timer = 0
189  202                       self.spawn_enemy()
190  203
191       -                # 敵の更新
192  204                   px, py = self.player.get_pos()
193  205                   for enemy in self.enemies:
194  206                       enemy.update(px, py)
195  207
196       -                # 当たり判定のチェック
197  208                   self.check_collisions()
198  209
199       -                # HPが0になったらゲームオーバー
200  210                   if self.player.get_hp() <= 0:
201  211                       self.game_state = "GAME_OVER"
202  212
203       -            # 描画処理
204  213               self.draw()
205       -
206  214               self.clock.tick(FPS)
207  215
208  216           pygame.quit()
209  217           sys.exit()
210  218
211  219       def spawn_enemy(self) -> None:
212       -        """画面の上下左右の端からランダムに敵を生成する"""
     220  +        """画面端からの敵の生成"""
213  221           side = random.randint(0, 3)
214       -        if side == 0:  # 上端
     222  +        if side == 0:
215  223               x = random.randint(0, SCREEN_WIDTH)
216  224               y = -20
217       -        elif side == 1:  # 下端
     225  +        elif side == 1:
218  226               x = random.randint(0, SCREEN_WIDTH)
219  227               y = SCREEN_HEIGHT + 20
220       -        elif side == 2:  # 左端
     228  +        elif side == 2:
221  229               x = -20
222  230               y = random.randint(0, SCREEN_HEIGHT)
223       -        else:  # 右端
     231  +        else:
224  232               x = SCREEN_WIDTH + 20
225  233               y = random.randint(0, SCREEN_HEIGHT)
226  234
227  235           self.enemies.append(Enemy(float(x), float(y)))
228  236
229  237       def check_collisions(self) -> None:
230       -        """プレイヤーとすべての敵の接触判定を行う"""
```

```
                + """当たり判定"""
231 239         px, py = self.player.get_pos()
232 240         p_radius = self.player.radius
233 241
236 244             e_radius = enemy.radius
237 245             distance = math.hypot(px - ex, py - ey)
238 246
239     -           # 円同士の当たり判定
240 247             if distance < p_radius + e_radius:
241     -               self.player.take_damage(20)  # 接触時に20ダメージ
                248 +               self.player.take_damage(20)
242 249
243 250     def draw(self) -> None:
244     -       """画面全体の描画を行う"""
245     -       # 背景描画
                251 +       """画面描画"""
246 252         self.screen.fill(COLOR_BG)
247 253
248     -       # キャラクター描画
249 254         self.player.draw(self.screen)
250 255         for enemy in self.enemies:
251 256             enemy.draw(self.screen)
252 257
253     -       # UI（HP、スコア、残り時間）の描画
                258 +       # UI描画
254 259         hp_text = self.font.render(f"HP: {self.player.get_hp()}/{self.player.max_hp}",
    True, COLOR_WHITE)
255 260         self.screen.blit(hp_text, (20, 20))
256 261
                262 +       # シールド機能のUI表示
                263 +       if self.player.shield_timer > 0:
                264 +           shield_str = f"Shield: ACTIVE ({self.player.shield_timer // FPS + 1}s)"
                265 +       elif self.player.shield_cooldown > 0:
                266 +           shield_str = f"Shield: CD ({self.player.shield_cooldown // FPS + 1}s)"
                267 +       else:
                268 +           shield_str = "Shield: READY (Press E)"
                269 +       shield_text = self.font.render(shield_str, True, COLOR_SHIELD)
                270 +       self.screen.blit(shield_text, (20, 55))
                271 +
257 272         survived_seconds = self.elapsed_frames // FPS
258 273         score_text = self.font.render(f"Score (Time): {survived_seconds}s", True,
    COLOR_WHITE)
259 274         self.screen.blit(score_text, (330, 20))
262 277         time_text = self.font.render(f"Limit: {remaining_seconds}s", True,
    COLOR_WHITE)
263 278         self.screen.blit(time_text, (730, 20))
264 279
265     -       # ゲーム終了時のオーバーレイとメッセージ
266 280         if self.game_state == "GAME_OVER":
267 281             overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT), pygame.SRCALPHA)
268 282             overlay.fill((0, 0, 0, 160))
```